

1. مقدمة

تم بناء العديد من الأدوات المساعدة في توليد برنامج التحليل المفرداتي. تقرأ هذه الأدوات توصيف كامل لمختلف مكونات اللغة (من كلمات مفتاحية ونماذج مُعرّفة باستخدام تعابير منتظمة) اعتباراً من ملف دخل، وتولد نص برمجي (بلغة C أو غيرها) قادر على مسح نص برمجي مصدري والتحقق من عدم وجود أي أخطاء في وفقاً للتوصيف المُعطى.

تم بناء أداة lex الخاصة بأنظمة Unix، وتم تطوير أداة مكافئة لها من قبل GNU وتدعى lex(f)، تأخذ ملفات دخل لها شكل محدد وتعطي برنامج بلغة C يمثل المحلل المفرداتي ويمكن ربطه ببقية أجزاء المترجم لبناء مترجم متكامل. يمكن تشغيل الأداة بالشكل:

```
> (f)lex specification_file.l
```

سيتولد نتيجة هذا التشغيل ملف lex.yy.c والذي يجب تمريره على مترجم للغة C مع المكتبة البرمجية الخاصة بالأداة lex(f):

```
> gcc lex.yy.c -l (f)l
```

يحتوي ملف التوصيف (specification_file.l) على تعابير منتظمة تعرف المفردات الممثلة لكل نموذج من نماذج الكلمات (متحول، عدد حقيقي، ... إلخ). يعمل المحلل، كما سبق وشرحنا، على تتبع كلمات النص البرمجي حرفاً، حرفاً حتى الوصول إلى أطول سلسلة من الأحرف متطابقة مع أحد التعابير المنتظمة أو أحد الرموز المعروفة.

2. بنية ملف توصيف المفردات

```
}%  
declaration (in C Language) of variables, constantes ,  
{%  
declaration of regular definitions  
%%  
Translation rules  
%%  
principal bloc & auxiliary functions (Optional)
```

تجدر الإشارة إلى أن القسم الأخير المتمثل في (principal bloc & auxiliary functions) هو قسم اختياري تتم إضافته في حال كان البرنامج الرئيسي (main) الخاص بالمحلل موجود في هذا الملف.

3. التعبيرات المنتظمة الخاصة بالأداة flex (f)

يتألف تعبير منتظم flex (f) من محارف عادية ومحارف ذات دلالة خاصة وهي:

\$, \, ^, [, { , < , > , - , + , * , / , | , ?

فيما يلي التعبيرات المنتظمة التي يقبلها flex (f) مع الانتباه إلى أن الأداة flex (f) حساسة للفرق بين الأحرف الصغيرة (small letters) والكبيرة (capital letters).

التعبير	ما تفهمه الأداة	مثال
c	حرف غير خاص	&
\c	في حال كان c محرف لا ينتمي إلى مجموعة الأحرف الصغيرة (small letters) من a إلى z، تأخذ الأداة الحرف c كما هو ودون أي اعتبارات خاصة وتعتبره حرف عادي غير خاص	\+
"s"	سلسلة المحارف s كما هي	"abc*+"
r ₁ r ₂	r ₁ متبوعة بـ r ₂	ab
^	في حال أتى كمحرف أول يسبق تعبيراً آخر، فإنه يُفهم على أن ما سيرد بعده يجب أن يكون في أول السطر	^ abc
\$	في حال أتى كمحرف أخير يلي تعبيراً آخر، فإنه يُفهم على أن التعبير الذي سيرد بعده سيكون في آخر السطر	abc\$
[s]	أي حرف من الأحرف المؤلفة للسلسلة s	[abc]
[^s]	أي حرف عدا الأحرف المؤلفة للسلسلة s	[^abc]
r*	يظهر التعبير r عدد من المرات تتراوح من 0 إلى اللانهاية	a*
r+	ظهور لمرة واحدة على الأقل للتعبير r	a+
r?	ظهور لمرة واحدة على الأكثر للتعبير r	a?

$a\{3\}$	ظهور m مرة للتعبير r	$r\{m\}$
$a\{2,5\}$	عدد مرات ظهور r يتراوح بين m و n	$r\{m,n\}$
$a \mid b$	r_1 أو r_2	$r_1 \mid r_2$
ab / cd	r_1 في حال كانت متبوعة بـ r_2	r_1 / r_2
$(a \mid b)? c$	r	(r)
	سطر جديد	$\backslash n$
	Tabulation	$\backslash t$
	نهاية الملف وهي خاصة بـ $(f)lex$	EOF

ملاحظات:

- لا يمكن للمحارف الخاصة $\$$ ، $\wedge$ ، $/$ الظهور ضمن قوسين () ولا ضمن تعاريف منتظمة.
- ضمن قوسين من الشكل []، يبقى المحرف \ فقط محرف خاص.
- لا يبقى المحرف - محرفاً خاصاً إلا إذا كان في المنتصف أي ليس في البداية ولا في النهاية.

4. المتحولات والإجراءات المعرفة مسبقاً في الأداة flex(f)

العنصر	الاستخدام
char yytext[]	جدول من المحارف يحتوي سلسلة الدخل التي تم قبولها
int yyleng	طول سلسلة الدخل التي تم قبولها
int yylex()	الإجرائية التي يجري عبرها إقلاع المحلل وتستدعي ()yywrap
int yywrap()	إجرائية يجري استدعاءها في نهاية سلسلة من محارف الدخل. تعتبر إجرائية مساعدة تعيد 0 في حال كان من الضروري متابعة التحليل (وجود ملف دخل آخر أو سلسلة دخل أخرى)، وتعيد 1 في الحالة المعاكسة
int main()	يحتوي فقط استدعاء للإجرائية ()yylex
unput(char c)	يحذف حرف من سلسلة الدخل بالعودة حرف إلى الوراء واعتبار الحرف غير مقروء (موجود فقط في flex(f) ولا يوجد في lex)
int yylineno	رقم السطر الحالي الذي تجري معالجته
yyterminate()	إيقاف التحليل (موجود فقط في flex(f) ولا يوجد في lex)

5. خيارات الترجمة

الخيار	المعنى
-d	حالة تنقيح (debug mode)
-i	عدم التفريق بين الأحرف الصغيرة (small letters) والأحرف الكبيرة (capital letters)
-l	للتوافق التام مع الأداة lex الخاصة بأنظمة unix
-s	للخروج مباشرة عند مصادفة أي محرف لا يتوافق مع أي رمز أو نموذج

6. نماذج عن ملف *.L

يقوم المحلل الموصف في الملف التالي بحساب عدد الأحرف الصوتية وعدد الأحرف غير الصوتية وعدد علامات التنقيط لسلسلة من المحارف المُدخلة من لوحة المفاتيح.

```
%{
    int nbvowels, nbconsonant, nbPunct;
}%
consonant      [b-df-hj-np-xz]
punctuation    [,:;?!\\.]
%%
[aeiouy]       nbvowels++;
{consonant}     nbconsonant++;
{punctuation}  nbPunct++;
.|\\n          // Nothing to do
%%
main(){
    nbvowels = nbconsonant = nbPunct = 0;
    yylex();
    printf("No %d vowels, %d consonants and %d
punctuations.\n",
           nbvowels, nbconsonant, nbPunct);
}
```

يقوم المحلل الموصف في الملف التالي بتعديل الدخل عبر حذف الأسطر التي تبدأ بالحرف p بالإضافة إلى حذف الأعداد الصحيحة، ومن ثم تحويل الحرف 0 إلى * والإبقاء على ما تبقى دون تغيير.

```
digit          [0-9]
integer         {digit}+
%%
{integer}      printf("I deny the integers");

^p(\\.)*\\n     printf("I hate lines starting with p\\n");

.              if (yytext[0]=='o') printf("*")
               else printf("%c",yytext[0]);
```